Wendi API Doc v1.0

Revision History

Date	Version	Author	Comments
17-03-2026	v1.0	Kennedy Gatimu	Initial document creation

Table of Contents

1.0 B2C Name Check API 4

Request URL 4
Request Field Description 4
Example Request Body 4
Success Response Body (200 OK) 5
Failure Response Body 5

2.0 B2C Transaction API 6

Request URL 6
Request Field Description 6
Example Request Body 7
Initial Response (Synchronous) 7
Final Response (Callback) 8
Failure Response 8

3.0 Authentication 9

Signature Validation 9
Implementation 9
Algorithm 9
Canonical Text Construction 9
Name Check (/wendi/outbound/b2c/namecheck) 9
Transaction (/wendi/outbound/b2c/transaction) 10
Java Signature Generation Sample 10
Private Key Loading Example 11

1.0 B2C Name Check API

This endpoint verifies a recipient wallet using MSISDN before initiating a transaction.
It returns wallet details to prevent failed or misdirected transfers.

Request URL

POST https://10.11.6.110:8443/wendi/outbound/b2c/namecheck

Request Field Description

Field	Type	Required	Description
receiver	Object	Mandatory	Contains recipient details
receiver.msisdn	String	Mandatory	Recipient mobile number. Pattern: ^\+?[0-9]+$, Length: 5–20
channelInfo	Object	Mandatory	Contains channel metadata
channelInfo.rrn	String	Mandatory	Retrieval Reference Number. Must be 12 characters, alphanumeric
channelInfo.bankId	String	Mandatory	Bank identifier. Pattern: [a-zA-Z0-9]+
channelInfo.channelId	String	Mandatory	Channel identifier. Pattern: [a-zA-Z0-9-]+
channelInfo.sourceSystemId	String	Mandatory	Source system identifier. Pattern: [a-zA-Z0-9-]+
signature	String	Mandatory	Digital signature. Max length: 1000 characters

Example Request Body


```
{
    "receiver": {
        "msisdn": "256783861192"
    },
    "channelInfo": {
        "rrn": "111996657000",
        "bankId": "54",
        "channelId": "OMNI",
        "sourceSystemId": "ONMI"
    },
    "signature":
"jZzbRrd8Ycq/WREA40ttMPtgbQoCmp/ZCL7Rmw8ulm/A2VEUvM14LEfn2jMzij1DWPzidsCkA6q7CYPPHLFIBV
pevEpTNp9WTm80RWYr8oZx5jT8i4FAFMnN58kXd1/QHfjgMxvfkP+T3XmFg2Kv5PdF7rOF0XvsaY9u6hfy2NbnQ
waSMg=="
}
```

Responses

A successful request returns a **200 OK** with the verified account holder's details.

An unsuccessful request will return an appropriate error code (4xx or 5xx) with a failure description.

Success Response Body (200 OK)


```
{
    "wendiResponse": {
        "statusCode": "00",
        "statusDescription": "Process service request successfully.",
        "walletName": "John Doe",
        "currency": "UGX",
        "allowCredit": "",
        "allowDebit": ""
    },
    "channelStatusInfo": {
        "status": "OK",
        "code": "00",
        "description": "COMPLETED_SUCCESSFULLY."
    }
}
```


Failure Response Body


```
{
    "channelStatusInfo": {
        "status": "FAILURE",
        "code": "01",
        "description": "Internal Error: HTTP call to the url was unsuccessful"
    } }
```

2.0 B2C Transaction API

This endpoint initiates a **B2C wallet transfer** from a sender account to a recipient wallet via Wendi.

The transaction is processed **asynchronously**:

- Initial response: **PENDING**
- Final result: delivered via **callback URL**

Request URL

POST https://10.11.6.110:8443/wendi/outbound/b2c/transaction

Request Field Description

Field	Type	Required	Description
sender	Object	Mandatory	Contains sender account details
sender.currency	String	Mandatory	3-letter currency code (e.g., UGX)
sender.accountNumber	String	Mandatory	Sender account number
sender.accountName	String	Mandatory	Sender account name
receiver	Object	Mandatory	Contains recipient wallet details
receiver.currency	String	Mandatory	3-letter currency code
receiver.accountNumber	String	Mandatory	Receiver account number
receiver.accountName	String	Mandatory	Receiver account name
receiver.msisdn	String	Mandatory	Recipient mobile number. Pattern: ^+?[0-9]+$, Length: 5–20
channelInfo	Object	Mandatory	Contains channel metadata
channelInfo.rrn	String	Mandatory	Retrieval Reference Number. Must be 12 characters, alphanumeric
channelInfo.bankId	String	Mandatory	Bank identifier. Pattern: [a-zA-Z0-9]+
channelInfo.channelId	String	Mandatory	Channel identifier. Pattern: [a-zA-Z0-9-]+
channelInfo.sourceSystemId	String	Mandatory	Source system identifier. Pattern: [a-zA-Z0-9-]+
channelInfo.callbackURL	String	Mandatory	URL for asynchronous response callback
transactionInfo	Object	Mandatory	Contains transaction details
transactionInfo.amount	String	Mandatory	Transaction amount (> 0)
transactionInfo.fee	String	Mandatory	Transaction fee (>= 0)
transactionInfo.transactionCurrency	String	Mandatory	Currency of transaction
transactionInfo.narration	String	Mandatory	Transaction description (max 100 characters)
signature	String	Mandatory	Digital signature (Base64 encoded, max 1000 characters)

Example Request Body

```
{
    "sender": {
        "currency": "UGX",
        "accountNumber": "0001100104442",
        "accountName": "John Doe"
    },
    "receiver": {
        "currency": "UGX",
        "accountNumber": "00000133800000",
        "accountName": "John Doe",
        "msisdn": "256783861192"
    },
    "channelInfo": {
        "rrn": "162996657000",
        "bankId": "54",
        "channelId": "OMNI",
        "sourceSystemId": "OMNI",
        "callbackURL": "https://testurl.co.ke"
    },
    "transactionInfo": {
        "amount": "100.00",
        "fee": "0",
        "transactionCurrency": "UGX",
        "narration": "Deposit to Wendi Account"
    },
    "signature": "<Base64EncodedSignature>"
}
```

Initial Response (Synchronous)

```
{
    "channelStatusInfo": {
        "status": "PENDING",
        "code": "-11",
        "description": "Transaction Processing Started. Final status via callback url"
    }
}
```

Final Response (Callback)


```
{
    "wendiResponse": {
        "statusCode": "00",
        "statusDescription": "Process service request successfully.",
        "requestId": "DES3242512344590UR898",
        "wendiReference": "BE4700876V"
    },
    "channelStatusInfo": {
        "status": "OK",
        "code": "00",
        "description": "COMPLETED_SUCCESSFULLY."
    }
}
```


Failure Response


```
{
    "channelStatusInfo": {
        "status": "FAILURE",
        "code": "01",
        "description": "Transaction Processing Failed: Generic error"
    }
}
```

3.0 Authentication

Wendi Service API exclusively utilizes a digital signature for authenticating all requests to ensure message **integrity, authenticity, and non-repudiation**.

Signature Validation

Applicable Endpoints:

- POST /wendi/outbound/b2c/namecheck
- POST /wendi/outbound/b2c/transaction

Implementation

The process involves creating a specific plain-text string (the "canonical text") from the request data, signing this text with your private key, and placing the Base64-encoded result into the signature field of the JSON body.

Algorithm

- SHA256withRSA
- Base64 encoded

Canonical Text Construction

The canonical text is a simple concatenation of specific field values from the request body. These values **must be appended in the exact order specified below, with no separators or extra characters.**

Name Check (/wendi/outbound/b2c/namecheck)

The canonical text is built by concatenating the following fields:

- channelInfo.rrn
- channelInfo.bankId
- channelInfo.channelId
- receiver.msisdn

Transaction (/wendi/outbound/b2c/transaction)

The canonical text is built by concatenating the following fields:

- channelInfo.rrn
- channelInfo.bankId
- channelInfo.channelId
- sender.accountName
- sender.accountNumber
- receiver.accountName
- receiver.accountNumber
- transactionInfo.amount
- transactionInfo.fee
- transactionInfo.transactionCurrency

Java Signature Generation Sample

```
public class WendiSignatureUtil {

    public static String generateSignature(TransactionRequest request, PrivateKey
privateKey) throws Exception {
        String clearText = buildCanonicalString(request);
        Signature signature = Signature.getInstance("SHA256withRSA");
        signature.initSign(privateKey);
        signature.update(clearText.getBytes(StandardCharsets.UTF_8));

        byte[] signedBytes = signature.sign();
        return Base64.getEncoder().encodeToString(signedBytes);
    }
    private static String buildCanonicalString(TransactionRequest req) {

        return req.getChannelInfo().getRrn()
                + req.getChannelInfo().getBankId()
                + req.getChannelInfo().getChannelId()
                + req.getSender().getAccountName()
                + req.getSender().getAccountNumber()
                + req.getReceiver().getAccountName()
                + req.getReceiver().getAccountNumber()
                + req.getTransactionInfo().getAmount()
                + req.getTransactionInfo().getFee()
                + req.getTransactionInfo().getTransactionCurrency();
    }
}
```

Private Key Loading Example

```
KeyStore keyStore = KeyStore.getInstance("PKCS12");
keyStore.load(new FileInputStream("keystore.p12"), "password".toCharArray());

PrivateKey privateKey = (PrivateKey) keyStore.getKey("alias",
"password".toCharArray());
```